

Same patient, different doctor — how are they not mixed up?

Explains how, even when the patient id is the same (e.g. 63514), a different doctor's data is **stored and queried separately**. Korean mirror: [SAME_PATIENT_DIFF_DOCTOR_KR.md](#).

Question: For two consultations with the **same patient id (same person) but different doctors**, can the system store and retrieve them without mixing them up?

Key 1. The record “key” is the file name, not the patient id

One file name carries patient · doctor · date:

```
63514 _ 63574 _ 06262026 . csv
^      ^      ^
patient(hash) doctor(hash) date
```

→ Same patient but **different doctor** ⇒ **different file name**. That file name IS the record's key (identity).

Key 2. Scenario — patient 63514 visits doctor A and doctor B

	📄📄 doctor A (63574) visit	📄📄 doctor B (13571) visit
file	63514_63574_0626.csv	63514_13571_0630.csv
patient id	63514 (same person)	63514 (same person)
→	file name differs (doctor·date) → different records	"

Key 3. Storage — different files ⇒ fully separate in the DB

📄📄 A visit stored	📄📄 B visit stored
transcript_analysis_log id=101 · patient_id=63514 · doctor_id=63574	id=102 · patient_id=63514 · doctor_id=13571
▼ tied by analysis_id=101	▼ tied by analysis_id=102
sentence_prediction / llm_domain_scoring_and_summary / ...	"
patient_summary file= 63514_63574_0626.csv	file= 63514_13571_0630.csv
all analysis_id=101	all analysis_id=102

Even with the same patient_id (63514), id·analysis_id·doctor_id·file differ → stored fully separately, no overwrite/collision.

Key 4. Retrieval — scoped precisely by file/doctor

📄📄 load A's	📄📄 load B's
?f=63514_63574_0626 → file → analysis_id=101 → A's results only	?f=63514_13571_0630 → analysis_id=102 → B's only
?doctorid=63574 → doctor_id filter → A's 63514	?doctorid=13571 → B's 63514

Same patient id (63514), but scoping by **file/doctor** means A and B are never mixed.

Key 5. What if we keyed on “patient id only”? (why it’s designed this way)

- **Patient id (63514) as the only key** : A and B visits share a key → a later visit overwrites/mixes the earlier one. You can't tell whether “63514's score” is A's or B's.
- **Actual design (file = patient+doctor+date key)**: each visit has a distinct key, kept separately. Plus a separate `doctor_id` column always says “whose”. You can still query “the whole patient (any doctor)” by `patient_id`, or “a specific doctor visit” by `file`.

Conclusion

The system distinguishes records by **file (patient+doctor+date)**, not **patient id**. So same-patient/different-doctor data is ① **stored separately** under different file/analysis_id, and ② **retrieved separately** via `?f=` (file)·`?doctorid=` (doctor).

(Caveat: some fallback code with a patient_id↔file mismatch can produce an empty result — not a mix-up — and is tracked separately for cleanup.)**